

PARADIGMAS DE LINGUAGENS DE PROGRAMAÇÃO

Prof. Rogerio Mandelli

ESCOLHA VIVER
MAIS EXPERIÊNCIAS.

Aula 04 | 20 mai 2026

UVA ●
A sua melhor escolha.

```
x = 10
y = 20

def soma(a):
    z = a + x
    return z

resultado = soma(y)
print(resultado)
```

BINDING

Associa identificadores
a valores na memória

Memória (dados)	
x	10
y	20
z	30
resultado	30

Execução (pilha de chamadas)

soma(a)
a = 20
z = 30

global
x = 10
y = 20
resultado = 30

Aula 4 - Binding, Escopo, Tipagem e Representação da Execução

Como linguagens representam, associam e controlam dados durante a execução do programa

TIPAGEM

Cada dado possui
um tipo

10	20	...
int	int	tipo

ESCOPO

Define onde os identificadores
são visíveis e acessíveis



Da Organização do Programa à Representação da Execução

Aula 3

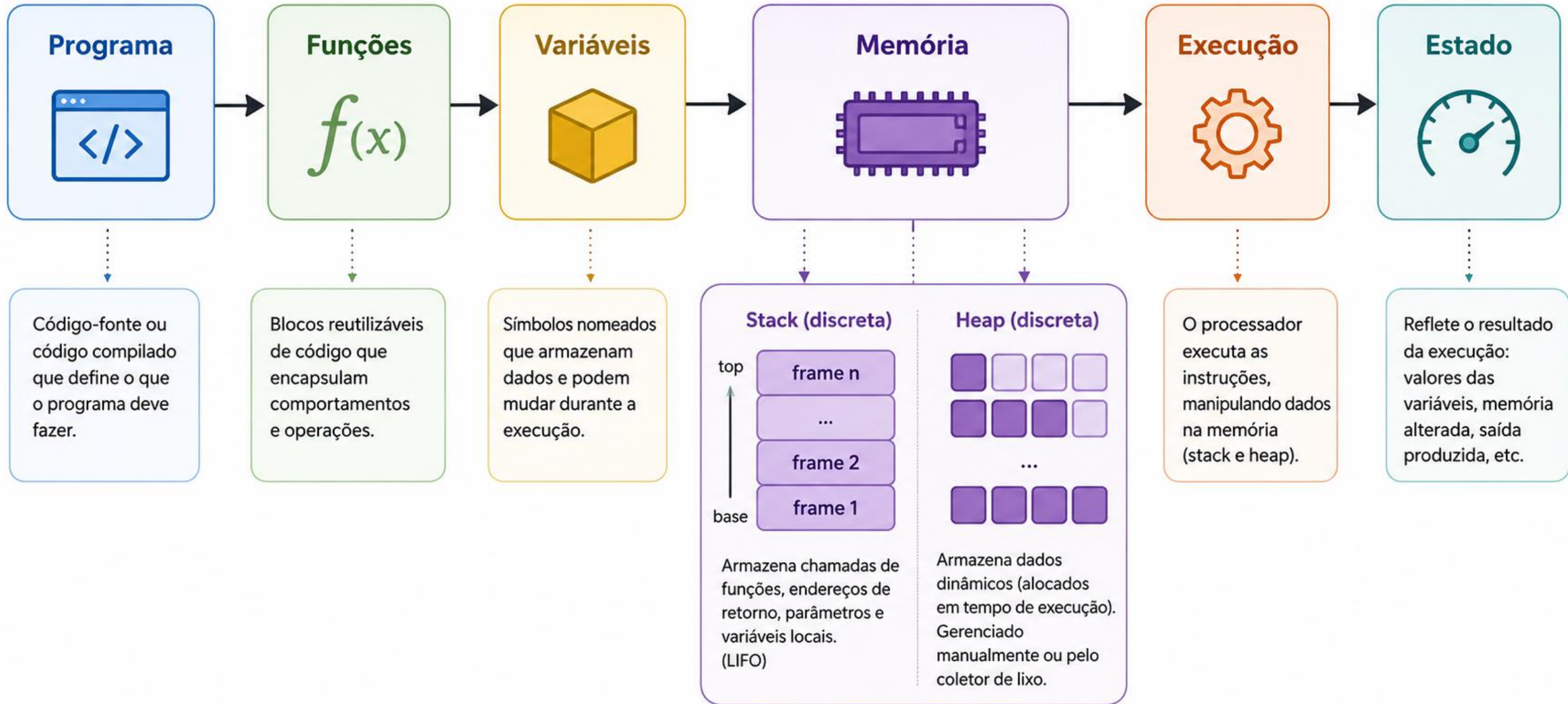
- modularização;
- separação de responsabilidades;
- funções e procedimentos;
- organização estrutural;
- encapsulamento introdutório;
- coordenação entre módulos

Aula 4

O que acontece com os dados durante a execução?

- > *Onde as variáveis existem?*
- > *Quando elas surgem e desaparecem?*
- > *Como linguagens associam nomes a valores?*
- > *Como memória e execução se relacionam?*
- > *Por que C e Python se comportam de forma diferente?*

Da Organização do Programa à Representação da Execução



Variáveis NÃO São Apenas “Caixas”

Modelo simplificado comum

“Variável é uma caixa que guarda um valor.”

✓ útil inicialmente

✗ insuficiente para entender linguagens

Simplificação não explica:

- escopo;
- referências;
- mutabilidade;
- *heap*;
- *stack*;
- contexto;
- tempo de vida;
- passagem de parâmetros;
- comportamento entre linguagens

Visão mais correta:

Uma variável envolve:

- nome;
- associação;
- tipo;
- valor;
- localização;
- contexto de execução;
- tempo de vida

Variáveis NÃO São Apenas “Caixas”

Exemplo simples

```
x = 10
```

```
x = x + 1
```

Perguntas:

→ *o valor mudou?*

→ *o nome mudou?*

→ *o objeto mudou?*

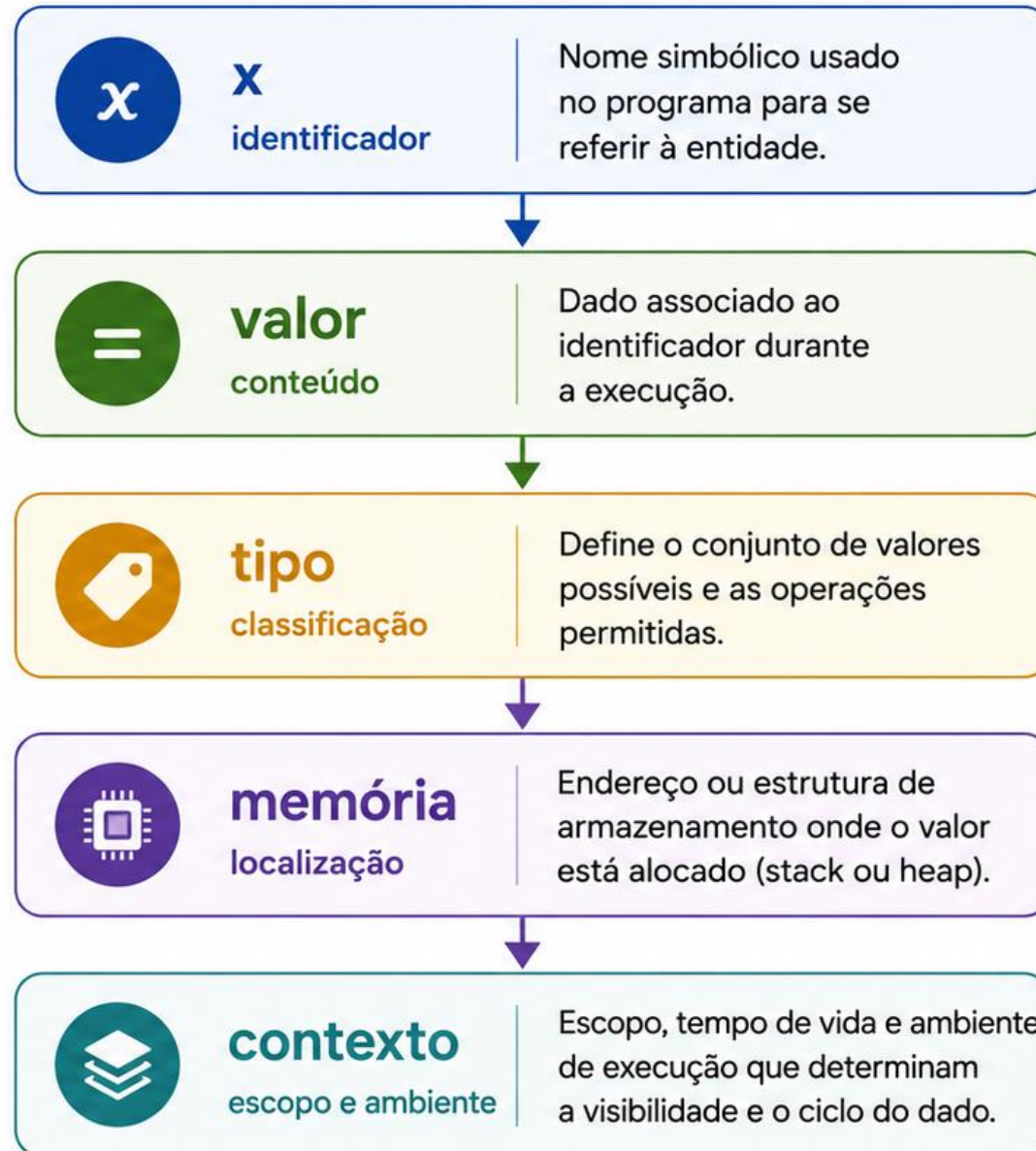
→ *o binding mudou?*

→ *onde isso está armazenado?*

Síntese conceitual

Linguagens não manipulam apenas valores, operam:

- > associações;
- > contexto;
- > representação;
- > identidade;
- > execução



Definição: *Binding*

Binding (amarração)

→ **associação** entre entidades do programa

Exemplos de *binding*

Associação	Exemplo
nome ↔ valor	<code>x = 10</code>
nome ↔ tipo	<code>int x</code>
nome ↔ endereço	variável na memória
chamada ↔ função	<code>soma(2,3)</code>
variável ↔ escopo	variável local/global

Ideia central

**Linguagens diferem
muito pela forma
como realizam
*bindings***

Definição: *Binding*

Comparação

Linguagem	Característica
C	bindings mais explícitos e rígidos
Python	bindings altamente dinâmicos
Java	bindings controlados pela JVM

Exemplo inicial

```
int x = 10
```

Bindings presentes:

nome ↔ tipo;

nome ↔ valor;

nome ↔ memória

Binding conecta:

→ semântica;

→ memória;

→ execução;

→ representação;

→ tipagem

Binding Estático × Dinâmico

***Binding* estático**

Associação ocorre: **antes da execução**

Exemplos:

- tipos em C;
- assinaturas;
- estruturas compiladas

***Binding* dinâmico**

Associação ocorre: **durante a execução**

Exemplos:

- variáveis em Python;
- referências;
- tipagem dinâmica

Comparação

Aspecto	Estático	Dinâmico
Flexibilidade	menor	maior
Segurança	maior	menor
Verificação	compilação	execução
Overhead	menor	maior

Binding Estático × Dinâmico

Exemplo comparativo

`</>` C

```
int x = 10;
```

`</>` Python

```
x = 10
```

```
x = "texto"
```

Discussão importante

O mesmo identificador pode representar entidades diferentes dependendo da linguagem

Escopo × Tempo de Vida

Conceito	Pergunta
Escopo	onde posso acessar
Tempo de vida	quando a variável existe

Exemplo

```
void teste() {  
  
    int x = 10;  
  
}
```

Interpretação

→ x só pode ser acessado dentro de `teste()`
→ x nasce quando a função inicia
→ x desaparece quando a função termina

Síntese

Escopo

→ visibilidade

Tempo de vida

→ existência durante a execução

Escopo × Tempo de Vida



Escopo e Modularização

Objetivo

- Conectar:
 - funções
 - escopo local
 - estado
 - encapsulamento
- Mensagem central
 - Funções também isolam contexto e estado

```
x = 100
```

```
def externa():  
    y = 50
```

```
def interna():  
    print(x)  
    print(y)
```

```
interna()
```

```
externa()
```

```
100
```

```
50
```

Chamadas de Função Criam Contextos

Durante a chamada

- a recebe o valor 2
- b recebe o valor 3
- resultado nasce dentro da função
- ao retornar, o contexto da função desaparece

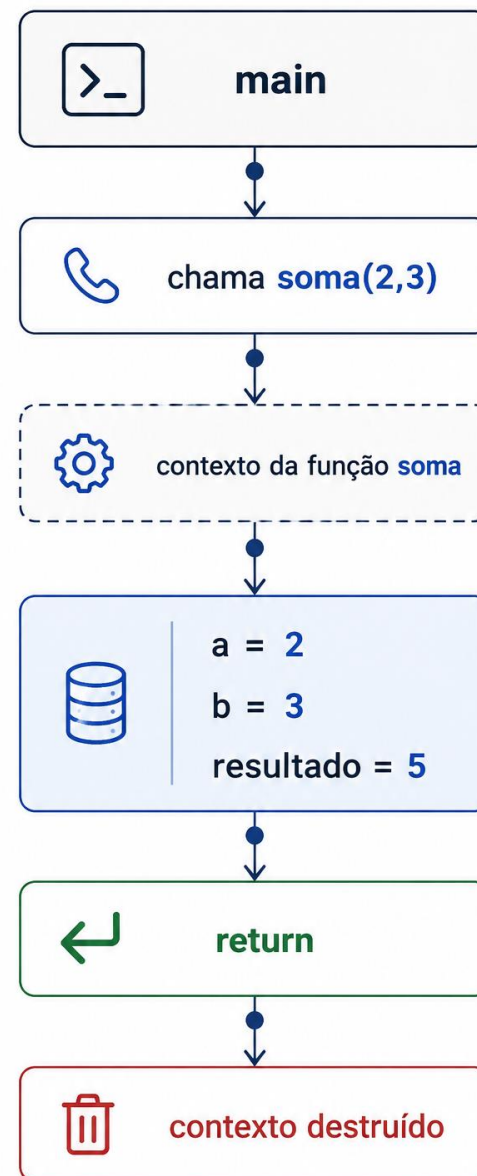
Pergunta central

Onde ficam a, b e resultado durante a execução?

`</>` Python

```
def soma(a, b):  
    resultado = a + b  
    return resultado
```

```
x = soma(2, 3)
```



ex_PLP_4.1.ipynb

Contexto de Execução e Chamadas de Função

Interpretação

- `soma()` cria um novo contexto de execução
- `a`, `b` e `resultado` existem apenas dentro de `soma()`
- `dobro()` cria outro contexto independente
- `x` e `y` existem apenas dentro de `dobro()`
- ao final da função, o contexto é destruído

```
def dobro(x):  
    y = x * 2  
    return y
```

```
def soma(a, b):  
    resultado = a + b  
    return dobro(resultado)
```

Cada chamada de função cria:

- variáveis locais próprias
- bindings temporários
- contexto temporário de execução

```
valor = soma(2, 3)
```

```
print(valor)
```

O que acontece durante uma chamada de função?

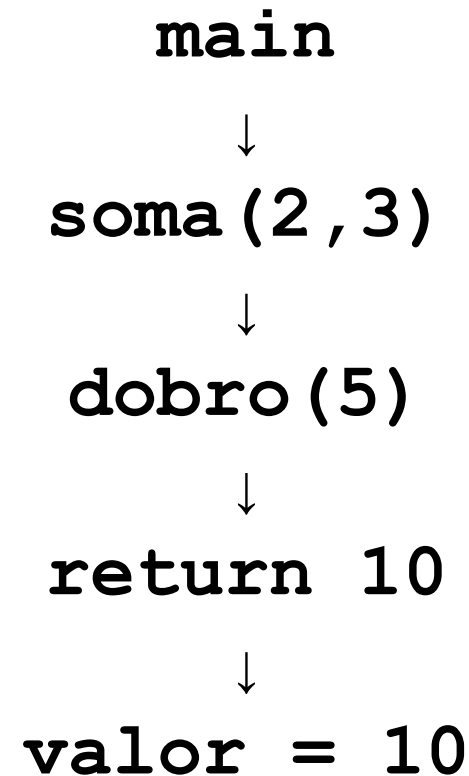
```
def dobro(x):  
    y = x * 2  
    return y
```

```
def soma(a, b):  
    resultado = a + b  
    return dobro(resultado)
```

```
valor = soma(2, 3)
```

```
print(valor)
```

ex_PLP_4.2.ipynb



Stack Frame (Visualização da Pilha) Status

Interpretação:

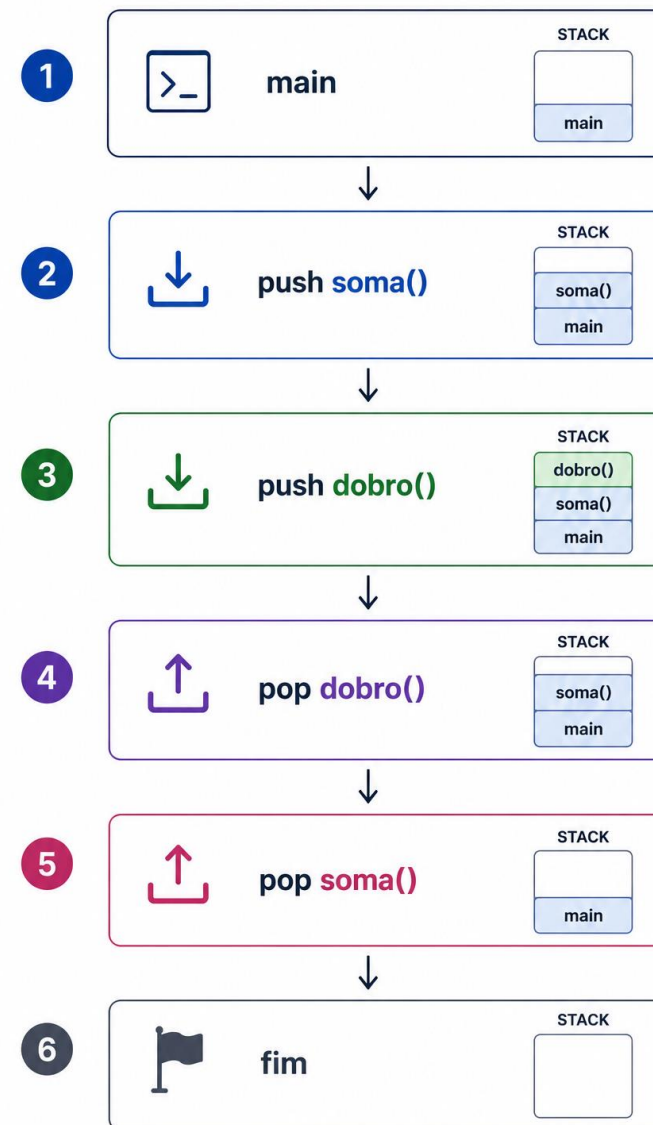
- cada bloco = stack frame
- topo = função atualmente executando
- variáveis locais pertencem ao frame atual



Push e Pop na Execução

Interpretação:

- **push**: novo contexto entra na stack
- **pop**: contexto removido após return
- execução entra e sai de contextos
- ordem da pilha importa
- última chamada é a primeira a terminar
- conexão com → LIFO (Last In First Out)



Ordem Real da Execução

```
main
↓
soma (2, 3)
  a = 2
  b = 3
resultado = 5
↓
dobro (5)
  x = 5
  y = 10
return 10
↓
return 10
↓
valor = 10
```

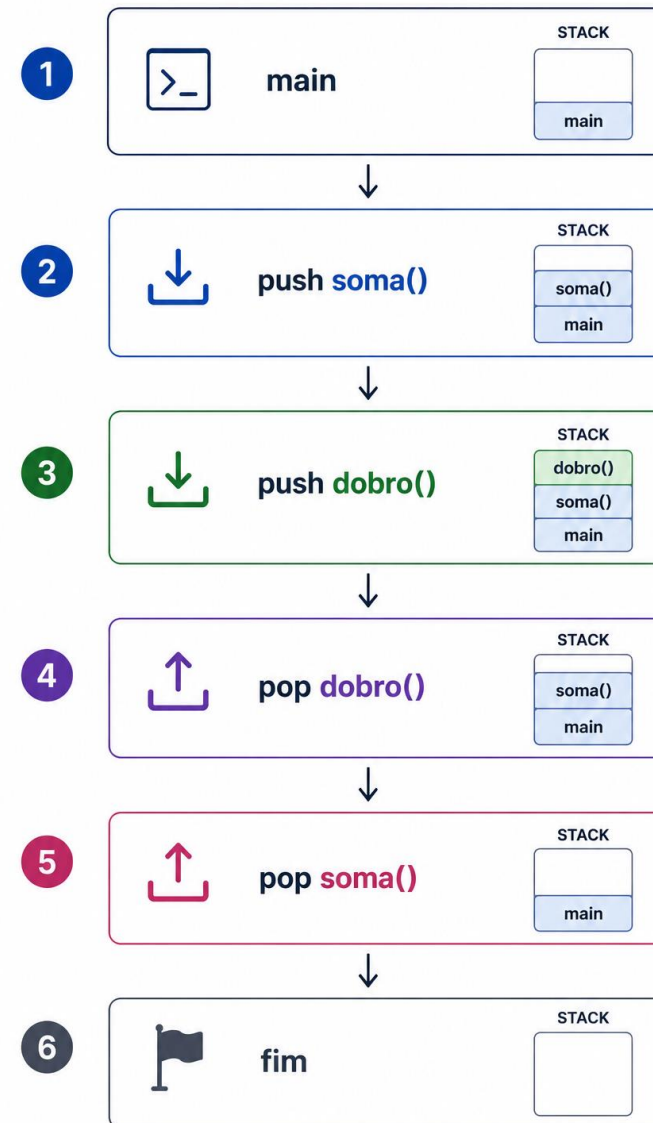
As variáveis de soma() continuam existindo após o return?

Resposta: Não.

O contexto local é destruído.

Relaciona:

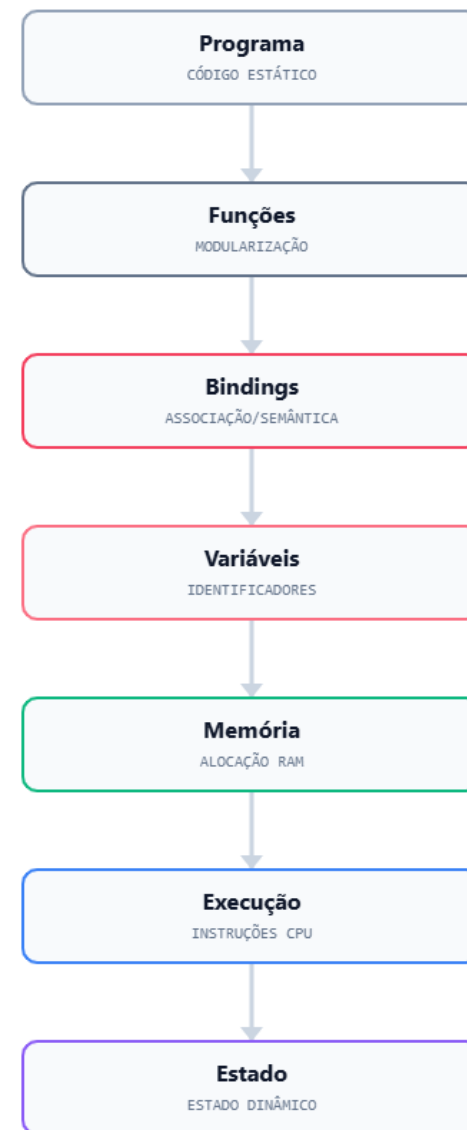
→ escopo local → tempo de vida → destruição automática do frame



Síntese do Contexto de Execução

Síntese:

- execução altera continuamente o estado do programa
- bindings surgem e desaparecem
- stack organiza chamadas e variáveis locais
- cada função possui contexto próprio



Da stack para a heap

Stack



controla chamadas
de função
↓
guarda contextos
locais temporários

Heap



guarda
objetos/dados que
podem sobreviver
ao contexto
imediato
↓
é acessada por
referências

Síntese:

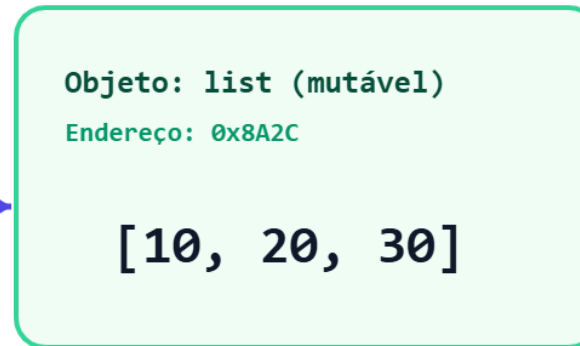
- variável não é necessariamente o dado
- variável pode guardar uma referência para um objeto
- em Python, listas, dicionários e objetos vivem como entidades referenciadas
- isso explica por que certos valores podem ser modificados sem criar nova variável

Referência não é cópia

STACK (Pilha de Execução)



HEAP (Memória Dinâmica)



```
a = [1, 2, 3]
```

```
b = a
```

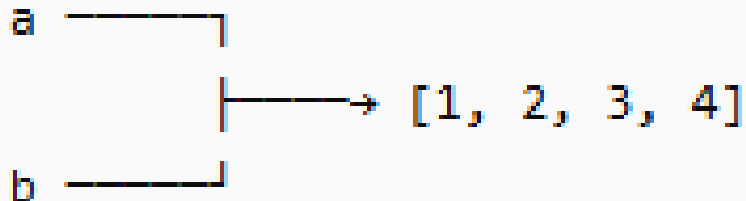
```
b.append(4)
```

```
print("a =", a)
```

```
print("b =", b)
```

```
print("id(a) =", id(a))
```

```
print("id(b) =", id(b))
```



Interpretação:

- a e b apontam para o mesmo objeto
- alterar via b afeta o objeto visto por a
- não houve cópia da lista
- houve cópia da referência

Cópia real: criando outro objeto

Saída esperada:

```
a = [1, 2, 3]
b = [1, 2, 3, 4]
id(a) = endereço lógico diferente
id(b) = endereço lógico diferente
```

```
a —————> [1, 2, 3]
b —————> [1, 2, 3, 4]
```

Interpretação:

- `copy()` cria outro objeto
- `a` e `b` deixam de compartilhar a mesma lista
- alterações em `b` não afetam `a`
- agora há dois objetos na heap

```
a = [1, 2, 3]
```

```
b = a.copy()
```

```
b.append(4)
```

```
print("a =", a)
```

```
print("b =", b)
```

```
print("id(a) =", id(a))
```

```
print("id(b) =", id(b))
```

Mutabilidade: mudar o objeto ou criar outro?

Comparação:

Tipo mutável



objeto pode ser alterado



lista, dict, set

Comparação:

Tipo imutável



operação cria novo objeto



int, float, str, tuple

```
x = 10
```

```
print("antes: ", x, id(x))
```

```
x = x + 1
```

```
print("depois:", x, id(x))
```

antes:

x —→ 10

depois:

x —→ 11

Interpretação:

→ 10 não vira 11

→ um novo objeto inteiro é associado ao nome x

→ o binding de x muda

→ o objeto anterior não foi modificado

Mutabilidade em listas: o objeto muda

Saída esperada:

```
antes:  [10, 20, 30]      mesmo id lógico  
depois: [10, 20, 30, 40] mesmo id lógico
```

```
antes:  
lista —————→ [10, 20, 30]  
  
depois:  
lista —————→ [10, 20, 30, 40]
```

```
lista = [10, 20, 30]  
print("antes: ", lista, id(lista))  
  
lista.append(40)  
print("depois:", lista, id(lista))
```

Síntese

- stack organiza nomes e contextos
- heap guarda objetos referenciados
- referências explicam compartilhamento
- mutabilidade explica efeitos inesperados

O que é tipagem?

Tipagem



define como
a linguagem
interpreta
e manipula
os dados

Perguntas fundamentais:

Esse valor é
inteiro?



texto?



real?



lista?



objeto?

```
x = 10
y = 3.14
nome = "Ana"
valores = [1, 2, 3]

print(type(x))
print(type(y))
print(type(nome))
print(type(valores))
```

Síntese:

- tipos determinam operações válidas
- tipos influenciam memória e execução
- linguagens tratam tipagem de maneiras diferentes

[ex_PLP_4.7.ipynb](#)

Tipagem dinâmica × estática

Comparação central:

Linguagem	Tipo declarado explicitamente?	Verificação
Python	normalmente não	dinâmica
Java	sim	estática
C	sim	estática

```
x = 10  
print(type(x))
```

```
x = "texto"  
print(type(x))
```

Interpretação:

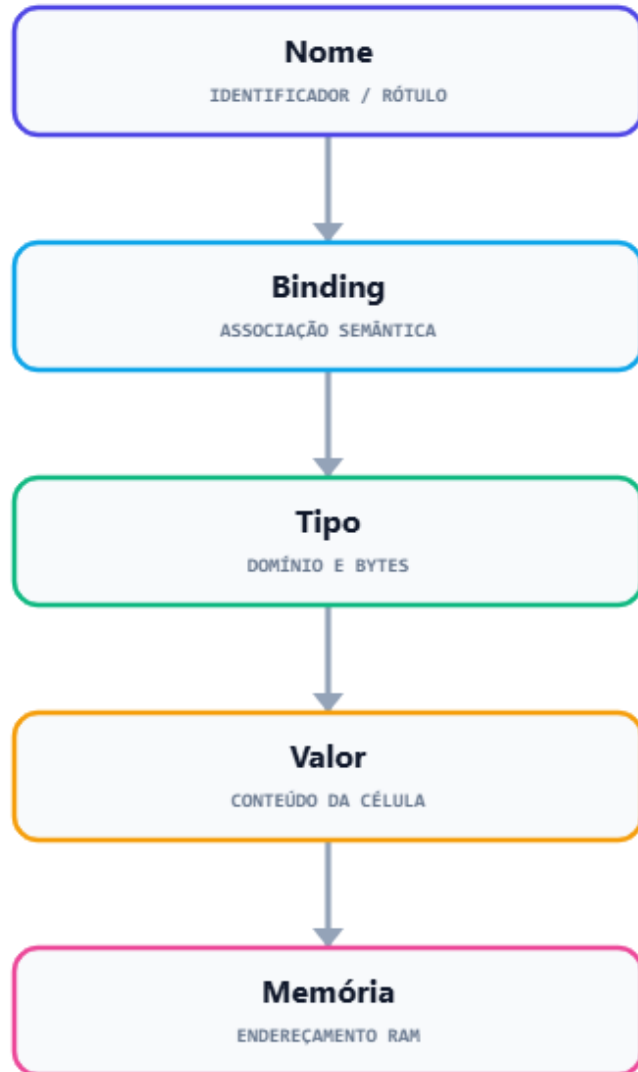
- Python permite rebinding com tipos diferentes
- C e Java exigem compatibilidade explícita
- tipagem influencia flexibilidade e segurança

ex_PLP_4.8.ipynb

```
⟨/⟩ C
```

```
int x = 10;  
x = "texto"; // erro
```

Representação dos dados na execução



Variável associa:

- nome
- tipo
- valor
- contexto

```
x = 10  
y = 10.5  
texto = "abc"
```

```
print("x =", x)  
print("tipo de x =", type(x))
```

```
print("y =", y)  
print("tipo de y =", type(y))
```

```
print("texto =", texto)  
print("tipo de texto =", type(texto))
```

Conversão de tipos e coerção

Conversão explícita:

```
x = "10"
```

```
numero = int(x)
```

```
print(numero + 5)
```

Interpretação:

- algumas conversões são automáticas
- outras precisam ser explícitas
- tipos incompatíveis geram erro semântico

Erro comum:

```
</> Python
```

```
x = "10"
```

```
print(x + 5)
```

Resultado:

TypeError

Exemplo em C:

```
</> C
```

```
int x = 10;
```

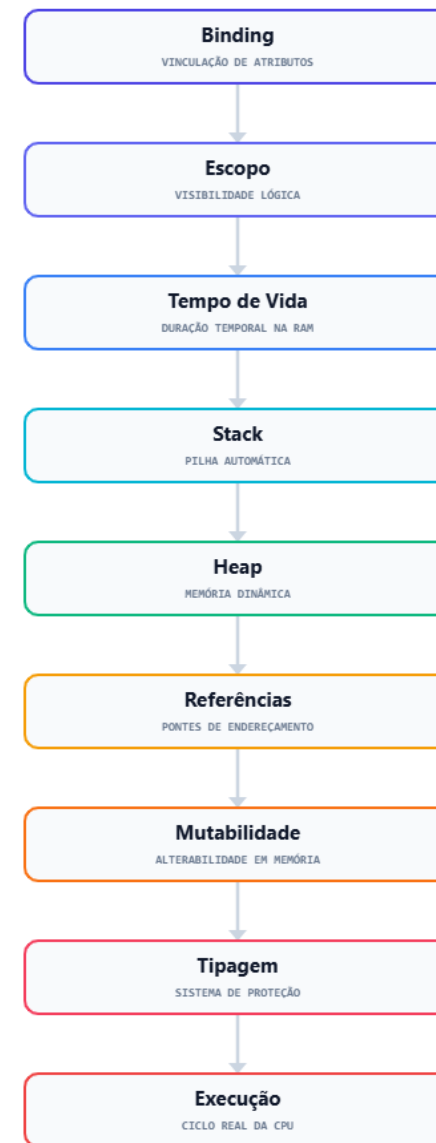
```
float y = 2.5;
```

```
float resultado = x + y;
```

Síntese da Unidade Semântica

- programas manipulam associações
- variáveis possuem contexto semântico
- objetos podem ser compartilhados
- tipos influenciam execução e segurança
- linguagens diferem na forma de representar dado

Como diferentes linguagens representam e controlam os mesmos dados?



Mesmo conceito, linguagens diferentes

Objetivo:

Comparar como C, Python e Java representam e manipulam dados durante a execução

Mesmo problema:

```
somar dois  
números  
↓  
armazenar  
resultado  
↓  
exibir saída
```

```
x = 10  
y = 20  
  
resultado = x + y  
  
print(resultado)
```

Mesmo conceito, linguagens diferentes

Objetivo:

Comparar como C, Python e Java representam e manipulam dados durante a execução

Mesmo problema:

```
somar dois  
números  
↓  
armazenar  
resultado  
↓  
exibir saída
```

ex_PLP_4.11.c

```
#include <stdio.h>

int main()
{
    int x = 10;
    int y = 20;

    int resultado = x + y;

    printf("%d\n", resultado);

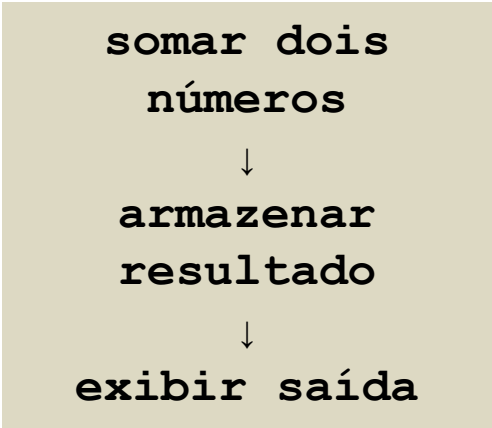
    return 0;
}
```

Mesmo conceito, linguagens diferentes

Objetivo:

Comparar como C, Python e Java representam e manipulam dados durante a execução

Mesmo problema:



```
graph TD; A[somar dois números] --> B[armazenar resultado]; B --> C[exibir saída];
```

somar dois
números
↓
armazenar
resultado
↓
exibir saída

```
public class Main
{
    public static void main(String[] args)
    {
        int x = 10;
        int y = 20;

        int resultado = x + y;

        System.out.println(resultado);
    }
}
```

Mesmo conceito, linguagens diferentes

Objetivo:

Comparar como C, Python e Java representam e manipulam dados durante a execução

Mesmo problema:

```
somar dois  
números  
↓  
armazenar  
resultado  
↓  
exibir saída
```

Síntese:

- mesmas ideias computacionais
- diferentes modelos semânticos
- diferentes níveis de abstração

Tipagem: dinâmica × estática

Linguagem	Tipagem
Python	dinâmica
C	estática
Java	estática

Python

→ flexibilidade

C/Java

→ controle semântico

Valor direto × referência

```
a = [1, 2]
b = a

b.append(3)

print('a = ', a)
print('b = ', b)
```

ex_PLP_4.12.ipynb

```
#include <stdio.h>

int main()
{
    int x = 10;
    int y = x;

    y = 20;

    printf("a = %d\n", x);
    printf("b = %d\n", y);

    return 0;
}
```

ex_PLP_4.12.c

Python

→ compartilhamento por referência

C

→ cópia direta do valor

Mutabilidade nas linguagens

```
lista = [1, 2]

print(id(lista))

lista.append(3)

print(id(lista))
```

```
#include <stdio.h>

int main()
{

    int x = 10;

    x = x + 1;

    printf("x = %d", x);

    return 0;
}
```

Síntese Comparativa Final

Aspecto	Python	Java	C
Tipagem	dinâmica	estática	estática
Objetos	tudo é objeto	parcialmente	manual
Referências	frequentes	objetos	ponteiros explícitos
Mutabilidade	intensa	parcial	depende do tipo
Execução	interpretada	JVM	compilada
Nível de abstração	alto	médio	baixo

Referências

- SEBESTA, R. W. Conceitos de Linguagens de Programação, Cap. 1–3.
- TUCKER & NOONAN. Linguagens de Programação: Princípios e Paradigmas.
- Vídeo: How Compilers and Interpreters Work – Computerphile.

